Feedback — Programming Homework 2

Help Center

You submitted this homework on **Sat 18 Jul 2015 10:45 PM PDT**. You got a score of **6.00** out of **6.00**.

In a practical, we saw Python code implementing the Boyer-Moore algorithm. Some of the code is for preprocessing the pattern P into the tables needed to execute the bad character and good suffix rules — we did not discuss that code. But we did discuss the code that performs the algorithm given those tables:

```
def boyer_moore(p, p_bm, t):
    """ Do Boyer-Moore matching. p=pattern, t=text,
        p_bm=BoyerMoore object for p """
    i = 0
    occurrences = []
    while i < len(t) - len(p) + 1:
        shift = 1
        mismatched = False
        for j in range(len(p)-1, -1, -1):
            if p[j] != t[i+j]:
                skip_bc = p_bm.bad_character_rule(j, t[i+j])
                skip_gs = p_bm.good_suffix_rule(j)
                shift = max(shift, skip_bc, skip_gs)
                mismatched = True
                break
        if not mismatched:
            occurrences.append(i)
            skip_gs = p_bm.match_skip()
            shift = max(shift, skip_gs)
        i += shift
    return occurrences
```

Measuring Boyer-Moore's benefit. First, download the Python module for Boyer-Moore preprocessing:

https://d28rh4a8wq0iu5.cloudfront.net/ads1/code/bm_preproc.py

This module provides the `BoyerMoore` class, which encapsulates the preprocessing info used by the `boyer_moore` function above. Second, download the provided excerpt of human chromosome 1:

https://d28rh4a8wq0iu5.cloudfront.net/ads1/data/chr1.GRCh38.excerpt.fasta

Third, implement versions of the naive exact matching and Boyer-Moore algorithms *that additionally count and return (a) the number of character comparisons performed and (b) the number of alignments tried*. Roughly speaking, these two counts measure how much work the two different algorithms are doing.

Question 1

How many alignments does the naive exact matching algorithm try when matching the string GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG (derived from human Alu sequences) to the excerpt of human chromosome 1?

You entered:

799954

Your Answer		Score	Explanation
799954	✔	1.00	
Total		1.00 / 1.00	

Question 2

How many character comparisons does the naive exact matching algorithm try when matching the string GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG (derived from human Alu sequences) to the excerpt of human chromosome 1?

You entered:

984143

Your Answer		Score	Explanation
984143	✔	1.00	
Total		1.00 / 1.00	

Question 3

How many alignments does Boyer-Moore try when matching the string GGCGCGGTGGCTCACGCCTGTAATCCCAGCACTTTGGGAGGCCGAGG (derived from human Alu sequences)

to the excerpt of human chromosome 1?

You entered:

127974

Your Answer		Score	Explanation
127974	✔	1.00	
Total		1.00 / 1.00	

Question 4

Index-assisted approximate matching. In practicals, we built a Python class called `Index` implementing an ordered-list version of the k-mer index. The `Index` class is copied below.

```
class Index(object):
    def __init__(self, t, k):
        ''' Create index from all substrings of size 'length' '''
        self.k = k  # k-mer length (k)
        self.index = []
        for i in range(len(t) - k + 1):  # for each k-mer
            self.index.append((t[i:i+k], i))  # add (k-mer, offset) pair
        self.index.sort()  # alphabetize by k-mer

    def query(self, p):
        ''' Return index hits for first k-mer of P '''
        kmer = p[:self.k]  # query with first k-mer
        i = bisect.bisect_left(self.index, (kmer, -1))  # binary search
        hits = []
        while i < len(self.index):  # collect matching index entries
            if self.index[i][0] != kmer:
                break
            hits.append(self.index[i][1])
            i += 1
        return hits
```

We also implemented the pigeonhole principle using Boyer-Moore as our exact matching algorithm.

Here we will implement the pigeonhole principle using `Index` to find exact matches for the partitions. We assume the pattern P always has length 24, and that we are always looking for approximate matches with up to 2 substitutions. We will use an 8-mer index.

Download the Python module for building a k-mer index.

https://d28rh4a8wq0iu5.cloudfront.net/ads1/code/kmer_index.py

Write a function that, given a length-24 pattern P and given an `Index` object built on 8-mers, finds all approximate occurrences of P within T with up to 2 mismatches. Insertions and deletions are not allowed.

How many times does the string `GGCGCGGTGGCTCACGCCTGTAAT` (derived from human Alu sequences) occur with up to 2 substitutions in the excerpt of human chromosome 1? Hint: multiple index hits might direct you to the same match multiple times; be careful not to count a match more than once.

You entered:

19

Your Answer		Score	Explanation
19	✔	1.00	
Total		1.00 / 1.00	

Question 5

Using the instructions given in Question 4, how many total index hits are there when searching for occurrences of `GGCGCGGTGGCTCACGCCTGTAAT` with up to 2 substitutions in the excerpt of human chromosome 1?

You entered:

90

Your Answer		Score	Explanation
90	✔	1.00	
Total		1.00 / 1.00	

Question 6

Let's examine whether there is a benefit to using an index built using *subsequences* of T rather than substrings. The following class `SubseqIndex` is a more general implementation of `Index` that additionally handles subsequences. It only considers subsequences that take every Nth character:

```
import bisect

class SubseqIndex(object):
    """ Holds a subsequence index for a text T """

    def __init__(self, t, k, ival):
        """ Create index from all subsequences consisting of k characters
            spaced ival positions apart.  E.g., SubseqIndex("ATAT", 2, 2)
            extracts ("AA", 0) and ("TT", 1). """
        self.k = k  # num characters per subsequence extracted
        self.ival = ival  # space between them; 1=adjacent, 2=every other, etc
        self.index = []
        self.span = 1 + ival * (k - 1)
        for i in range(len(t) - self.span + 1):  # for each subseq
            self.index.append((t[i:i+self.span:ival], i))  # add (subseq, offset)
        self.index.sort()  # alphabetize by subseq

    def query(self, p):
        """ Return index hits for first subseq of p """
        subseq = p[:self.span:self.ival]  # query with first subseq
        i = bisect.bisect_left(self.index, (subseq, -1))  # binary search
        hits = []
        while i < len(self.index):  # collect matching index entries
            if self.index[i][0] != subseq:
                break
            hits.append(self.index[i][1])
            i += 1
        return hits
```

For example, if we do:

```
ind = SubseqIndex('ATATAT', 3, 2)
print(ind.index)
```

we see:

```
[('AAA', 0), ('TTT', 1)]
```

And if we query this index:

```
p = 'TTATAT'
print(ind.query(p[0:]))
```

we see:

```
[]
```

because the subsequence "TAA" is not in the index. But if we query with the second subsequence:

```
print(ind.query(p[1:]))
```

we see:

```
[1]
```

because the second subsequence "TTT" is in the index.

Write a function that, given a length-24 pattern P and given a SubseqIndex object built with k = 8 and ival = 3, finds all approximate occurrences of P within T with up to 2 mismatches.

When using this function, how many total index hits are there when searching for GGCGCGGTGGCTCACGCCTGTAAT with up to 2 substitutions in the excerpt of human chromosome 1?

You entered:

79

Your Answer		Score	Explanation
79	✔	1.00	
Total		1.00 / 1.00	